



南开大学
Nankai University

南 开 大 学

数据安全实验报告

实验 2：零知识证明实践

学院：密码与网络空间安全学院

专业：物联网工程

学号：2310422

姓名：谢小珂

指导教师：刘哲理

2026 年 5 月 5 日

目录

1 实验要求	1
2 实验环境	1
2.1 软件架构与依赖	1
2.2 环境安装命令	1
2.3 实验代码目录结构	2
2.4 编译与构建指令	2
2.5 运行配置说明	2
3 实验原理	2
3.1 整体流程架构	3
3.2 核心数学工具	4
3.3 目标方程算术化与约束电路构造	4
3.3.1 中间变量引入与方程拆解	4
3.3.2 R1CS 约束描述	4
3.3.3 变量边界划分	5
4 实验代码分析	5
4.1 common.hpp 代码逻辑分析	5
4.1.1 核心流程推理	5
4.1.2 关键代码片段解析	5
4.1.3 核心作用总结	6
4.2 mysetup.cpp 代码逻辑分析	6
4.2.1 核心流程推理	6
4.2.2 关键代码片段解析	6
4.2.3 安全性总结	7
4.3 myprove.cpp 代码逻辑分析	7
4.3.1 核心流程推理	7
4.3.2 关键代码片段解析	8
4.3.3 安全性总结	8
4.4 myverify.cpp 代码逻辑分析	8
4.4.1 核心流程推理	8
4.4.2 关键代码片段解析	9
4.4.3 安全性总结	9
5 实验步骤与结果分析	9
5.0.1 密钥生成阶段 (Setup)	9
5.0.2 证明生成阶段 (Prove)	10
5.0.3 证明验证阶段 (Verify)	11
6 实验遇到的问题	11
7 实验总结	12

1 实验要求

参考教材实验 3.1, 假设 Alice 希望证明自己知道如下方程的解

$$x^3 + x + 5 = out$$

其中 out 是公开已知常数, 本题中设定 $out = 35$, 该方程的真实解为 $x = 3$ 。

请编写可运行代码, 完成零知识证明的证明生成与证明验证全流程实现。

2 实验环境

2.1 软件架构与依赖

本实验基于 Linux 操作系统, 利用 C++ 语言结合 libsnark 密码学库实现。零知识证明的实现依赖于多项底层数学运算库, 主要环境要求如下:

操作系统: Ubuntu 18.04 / 20.04 LTS 或更高版本的 Linux 系统。

编译器: 支持 C++11/C++14 标准的 g++ (建议版本 7.5+)。

构建工具: CMake (版本 3.10+)。

核心库:

libsnark: 零知识证明协议栈的核心库。

libff: 用于椭圆曲线、有限域等基础代数运算。

GMP (GNU Multiple Precision Arithmetic Library): 高精度数学运算库, 用于处理大整数。

Boost: 提供系统、文件系统等基础支持。

OpenSSL: 用于哈希运算和伪随机数生成。

2.2 环境安装命令

在实验开始前, 需要在终端中顺序执行以下命令以部署实验环境:

更新系统包列表:

```
1 sudo apt-get update
```

安装必要的基础库与工具:

```
1 sudo apt-get install build-essential git libgmp3-dev libprocps-dev libgtest-  
dev libboost-all-dev libssl-dev cmake
```

获取并安装 libsnark (若实验环境未预装):

```
1 git clone --recursive https://github.com/scipr-lab/libsnark.git  
2 cd libsnark  
3 mkdir build && cd build  
4 cmake ..  
5 make  
6 sudo make install
```

注: 实验代码 CMakeLists.txt 中已配置 `link_directories(/usr/local/lib)`, 因此需确保库文件安装至 `/usr/local/lib`。

2.3 实验代码目录结构

在实验目录下，应包含以下文件结构：

```
.
CMakeLists.txt      # 自动化编译配置文件
common.hpp          # 定义 R1CS 约束的核心头文件
mysetup.cpp         # 密钥对生成程序
myprove.cpp         # 证明生成程序
myverify.cpp        # 验证程序
(编译后生成 bin/ 或相关可执行文件)
```

2.4 编译与构建指令

利用 CMake 进行项目的统一管理，执行以下步骤完成代码编译：

初始化构建目录：

```
1 mkdir build && cd build
```

运行 CMake 生成 Makefile：

```
1 cmake ..
```

此处 CMake 会检查系统中是否安装了 OpenSSL、Threads 和 Boost，并自动设置 CURVE_ALT_BN128 椭圆曲线参数（这是 libsnark 最常用的曲线）。

执行编译：

```
1 make
```

编译完成后，目录下会生成三个可执行文件：mysetup、myprove 和 myverify。

2.5 运行配置说明

在运行阶段，各程序通过读写二进制文件进行交互：

pk.raw / vk.raw：由 mysetup 产生，存储证明/验证密钥。

proof.raw：由 myprove 产生，存储零知识证明。

标准输入流：程序运行过程中需手动输入公开值 out (35) 或私密解 x (3)。

3 实验原理

本实验采用 zk-SNARKs（简洁非交互式零知识知识论证）技术实现方程知识的证明，核心原理是将目标代数计算问题标准化拆解、编译转化为密码学可验证约束电路，依托前沿密码学底层算子完成可信轻量化证明交互，全程不泄露任何私密求解信息，适配本实验自定义代数方程场景下的轻量化零知识核验需求。

3.1 整体流程架构

整体过程如下图所示：

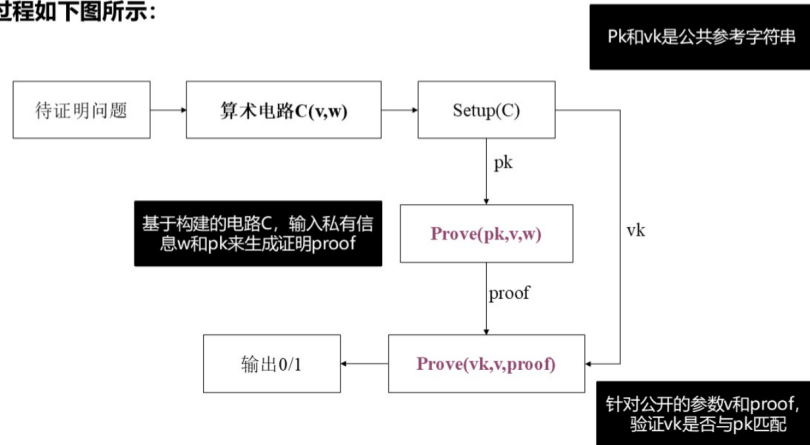


图 1: zk-SNARKs 系统的通用操作模型

结合图 1 完整链路架构，zk-SNARKs 整套加密证明体系可拆解为四大联动核心执行环节，贴合本实验方程证明业务逻辑落地运行，分步原理说明如下：

待证明问题与算术电路转化 (Arithmetization)：作为全流程前置核心预处理环节，核心任务是将本次实验目标数学方程 $x^3 + x + 5 = out$ 完整拆解、映射转化为标准化可解析算术电路 $C(v, w)$ 。其中，参数核心定义明确区分公私两类数据： v (Public Input) 为全局公开输入参数，对应方程最终运算结果 out ，全网所有参与核验节点均可读取； w (Witness) 为专属私密见证数据，仅由证明者单独持有，对应方程唯一有效私密解 x 。复杂非线性方程会被逐层拆分拆解为加法、乘法两类基础逻辑门电路，同步规整编码生成密码学通用的一阶秩约束系统，即 R1CS (Rank-1 Constraint System)。贴合本实验实测场景，方程标准化拆解约束链路如下： $g_1 : x \cdot x = x_sq$ 、 $g_2 : x_sq \cdot x = x_cu$ 、 $g_3 : 1 \cdot (x_cu + x + 5) = out$ ，完成全量电路约束闭环搭建。

设置阶段 (Setup)：

依托前端已闭环构建的完整算术电路 C ，依托 libsnark 底层密码学算子批量生成全局公共参考字符串 (CRS)，为后续证明、核验全流程提供合规可信公共参数支撑。阶段固定输入为标准化结构化电路完整描述文件 C ，定向输出两类配套核心密钥参数，分别为专属证明密钥 pk (Proving Key)、权威验证密钥 vk (Verification Key)。该阶段生成的全部公共密钥参数具备一次生成、全域复用特性，单次部署完成后，全链路证明者、验证者均可反复调用，无需重复初始化配置，大幅降低实验整体算力开销。

证明阶段 (Prove)：由证明者 Alice 独立闭环完成轻量化证明生成操作，全程离线自主运算，无任何私密数据外溢风险。核心运算输入包含三类合规参数，分别为前置初始化生成的证明密钥 pk 、全网同步公开常量参数 v 、仅本地留存的私密见证原始数据 w (本实验固定私密求解值 $x = 3$)。核心运算过程为：证明者依托 libsnark 内置多项式编码算法、椭圆曲线映射算子，批量求解电路全链路中间运算变量，压缩加密后封装生成字节体量极小的标准化零知识证明凭证 $proof$ 。全程严格遵循零知识核心安全特性，加密封装后的证明凭证不会回溯泄露私密见证 w 的任何原始特征，外部恶意节点无法逆向推导破解私密解 x ，筑牢实验数据隐私安全防线。

验证阶段 (Verify)：由验证者 Bob 低成本快速完成凭证合规性核验，无需复杂算力运算，核验效率高效便捷。核验固定输入包含权威验证密钥 vk 、全网统一公开参数 v 、证明者上传的加密零知识证明 $proof$ 。系统自动执行配对校验运算，最终输出二元核验结果 0 或 1，1 代表方程合规、证明有效，0 代表数据篡改、证明失效。该阶段兼具双重核心优势，一是简洁性，整体核

验算力开销极低、响应速度快；二是非交互性，全程无需证明者 Alice 实时在线联动交互，离线即可完成全量核验闭环。

3.2 核心数学工具

双线性配对 (Bilinear Pairing)：是 libsnark 密码库底层支撑证明核验有效性的核心基础数学算子，也是整套 zk-SNARKs 机制落地运行的底层核心底座。依托椭圆曲线定点高效点运算逻辑，无需读取、解析任何私密原始数据，即可跨链路闭环校验全量电路约束等式成立性，从数学底层保障证明不可伪造、核验安全可信，适配本实验轻量化落地场景。

R1CS 到 QAP 的转化：实验实操阶段可直接调用接口编辑、配置 R1CS 约束规则，无需手动参与底层算法优化；后台 libsnark 库会自动完成深度二次转化，将结构化 R1CS 约束体系批量编译映射为二次算术程序 (QAP)。依托多项式全域插值、多项式匹配校验天然数学特性，压缩优化证明生成与核验全链路算力损耗，兼顾实验实操便捷性与零知识证明整体运行高效性。

3.3 目标方程算术化与约束电路构造

在零知识证明实验中，待证明的命题通常以多项式方程的形式给出，例如：

$$x^3 + x + 5 = out$$

其中 out 为公开输入 (Public Input)， x 为证明者拥有的私密见证 (Private Witness)。

由于 libsnark 等证明框架无法直接处理高次多项式，必须将其“算术电路化”，即拆解为一组一阶约束系统 (R1CS)。R1CS 的每一条约束都必须满足以下标准形式：

$$\langle a, z \rangle \cdot \langle b, z \rangle = \langle c, z \rangle$$

这意味着每条约束只能包含一次乘法关系（线性组合的乘积等于另一个线性组合）。

3.3.1 中间变量引入与方程拆解

为了满足 R1CS 的要求，本实验引入中间变量 sym_1 、 y 和 sym_2 ，将原始的三次方程拆解为以下四个计算步骤：

1. 计算平方项： $sym_1 = x \cdot x$
2. 计算立方项： $y = sym_1 \cdot x$
3. 计算加法项： $sym_2 = y + x$
4. 验证最终结果： $out = sym_2 + 5$

3.3.2 R1CS 约束描述

上述计算步骤在证明系统中被表达为四条核心约束：

编号	R1CS 约束表达式	物理含义
1	$x \cdot x = sym_1$	计算 x^2 ，为构造三次项做准备
2	$sym_1 \cdot x = y$	计算 x^3
3	$(y + x) \cdot 1 = sym_2$	执行加法运算，合并三次项与一次项
4	$(sym_2 + 5) \cdot 1 = out$	引入常数项并要求结果等于公开输出 out

3.3.3 变量边界划分

在约束系统中，变量被严格划分为两类，以确保隐私性：

- **公开输入 (Primary Input)**：验证者已知的数值。本实验中仅 *out* 被声明为公开输入（例如 $out = 35$ ）。
- **辅助输入 (Auxiliary Input)**：仅证明者可见的私密信息。包括秘密值 x 以及由其推导出的中间变量 sym_1, y, sym_2 。

通过这种约束设计，证明系统能够检查 witness 内部的逻辑自洽性：如果证明者提供的中间变量不符合上述乘法或加法约束，`is_satisfied()` 检查将失败，从而无法生成有效的证明文件。

4 实验代码分析

4.1 common.hpp 代码逻辑分析

4.1.1 核心流程推理

`common.hpp` 是整个零知识证明实验的公共底层头文件，统一规范全局算术电路拓扑结构，核心作用是将非线性高次目标方程标准化拆解、编译为合规 R1CS 秩一约束系统，为密钥生成、证明构造、全量核验全流程提供统一电路底层基准支撑。

文件首先完成底层密码学环境类型统一声明，通过类型别名绑定适配当前实验椭圆曲线对应的有限标量域，全局锁定电路内所有加法、乘法数值运算规则，确保全链路密码运算格式统一、兼容适配，规避域参数错乱导致链路失效问题。

随后执行全局电路变量规范化布局分配，依托 `libsnark` 专属协议板依次有序注册公开输出变量、私密求解变量及两类中间过渡运算变量，固定全局变量排布索引顺序，变量排布逻辑直接界定后续公开输入与私密见证的边界划分基准，是公私数据隔离的核心前置前提。

精准划定公私变量隔离边界，通过专属接口硬性标定首位变量为全局公开可读参数，剩余全部求解类、中间类变量统一归类为私密不可见见证数据，从代码底层严格切分公私数据权限，贴合零知识隐私隔离核心规范。最后完成目标方程全量算术化约束拆解，贴合 zk-SNARKs 强制 $A \times B = C$ 标准约束格式，逐行拆解三次非线性目标方程，拆分生成三组合规线性约束规则，批量写入协议板闭环成型完整电路约束链路，筑牢全流程可信核验底层逻辑根基。

4.1.2 关键代码片段解析

```

1 // 1. 变量实例化：在协议板上注册所有数学占位符
2 out.allocate(pb, "out");
3 x.allocate(pb, "x");
4 x_sq.allocate(pb, "x_sq");
5 x_cu.allocate(pb, "x_cu");
6
7 // 2. 边界划分：指定前 1 个分配的变量为 Public Input
8 pb.set_input_sizes(1);
9
10 // 3. 约束构造：将高次方程线性化为  $A * B = C$  形式
11 // 约束 1:  $x * x = x\_sq$ 
12 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, x, x_sq));
13

```

```

14 // 约束 2:  $x_{sq} * x = x_{cu}$ 
15 pb.add_rlcs_constraint(rlcs_constraint<FieldT>(x_sq, x, x_cu));
16
17 // 约束 3:  $1 * (x_{cu} + x + 5) = out$ 
18 // 利用常量 1 和加法组合实现线性累加验证
19 pb.add_rlcs_constraint(rlcs_constraint<FieldT>(1, x_cu + x + const_5, out));

```

4.1.3 核心作用总结

common.hpp 是整套零知识证明实验链路的统一核心共识基底，依托标准化电路构建函数，强制约束密钥生成、证明构造、结果核验三大核心程序复用完全一致的电路拓扑、变量排布与约束规则。全程统一底层逻辑标准，保障全链路数据联动适配、无偏差衔接，若该头文件内任意约束公式、变量排布参数发生改动，整套闭环证明链路会直接校验失效，无法正常完成密钥生成与合规核验，是保障实验全流程稳定可信运行的核心底层支撑文件。

4.2 mysetup.cpp 代码逻辑分析

4.2.1 核心流程推理

mysetup.cpp 作为整套零知识证明实验的前置核心程序，核心功能是依据预设算术约束电路，标准化生成配套可信证明密钥 pk 与验证密钥 vk，为后续证明、核验全流程提供合规公共密码参数支撑，整体闭环执行逻辑分为四大联动核心环节。

首先进行公共参数全局初始化，程序主动调用配套底层接口完成预设椭圆曲线环境部署，本实验统一适配适配性更强、安全性达标的 BN128 标准曲线，全局敲定后续所有群运算、双线性配对运算的底层数学基底，保障全链路密码运算合规可控。

随后完成标准化约束系统实例化搭建，依托专属协议板工具完成全局变量索引规整分配，全程无需提前赋值私密解与公开参数，仅结构化梳理方程拓扑关联逻辑，同步将目标实验方程 $x^3 + x + 5 = out$ 自动拆解转化为多组标准 $A \cdot B = C$ 结构化 R1CS 线性约束矩阵，完整固定问题电路拓扑框架，不录入任何实测运算数据。

接着执行核心密钥对生成运算，调用零知识证明标准 Setup 底层算法，将闭环 R1CS 约束系统作为唯一输入，在合规椭圆曲线群内生成随机安全基准参数，拆分输出两类专用密钥：证明密钥 pk 完整收录全电路门电路加密映射信息，支撑证明者隐私化拼接多项式、构造合规零知识证明；验证密钥 vk 轻量化精简提纯核心校验参数，仅保留电路一致性核验必备椭圆曲线点位，适配低成本快速核验场景。

最后完成密钥二进制序列化持久化存储，将批量高精度椭圆曲线点位密钥数据，以原生二进制格式写入 pk.raw 与 vk.raw 本地文件，既完整保全密钥原始运算精度、杜绝数据失真，又大幅提升后续读写调用效率，适配本地实验离线复用需求。

4.2.2 关键代码片段解析

程序核心执行逻辑高度凝练，关键功能性代码如下所示，分步对应上述全流程核心操作，适配 libsnark 库标准调用规范：

```

1 // 1. 初始化全局公共参数，部署BN128椭圆曲线底层运行环境
2 default_rlcs_gg_ppzksnark_pp::init_public_params();
3
4 // 2. 结构化搭建算术电路约束框架，仅构建逻辑拓扑，不填充实测数值
5 build_protoboard(pb, x, x_sq, x_cu, out);

```



```

6  const rlcs_constraint_system<FieldT> constraint_system = pb.
    get_constraint_system();
7
8  // 3. 调用底层Setup算法，依托R1CS约束批量生成公私配套密钥对
9  auto keypair = rlcs_gg_ppzksnark_generator<default_rlcs_gg_ppzksnark_pp>(
    constraint_system);
10
11 // 4. 二进制持久化写入证明密钥，留存供证明、核验程序联动调用
12 ofstream pk_file("pk.raw", ios::binary);
13 pk_file << keypair.pk;

```

4.2.3 安全性总结

mysetup.cpp 密钥生成阶段随机采样生成的底层私密随机参数，行业内统一称为有毒废物 (Toxic Waste)，是整套前置初始化流程的安全核心命脉。在正式商用落地场景中，若该核心私密随机参数被恶意非法截留、窃取，攻击者可绕过合规约束校验，随意伪造任意合法零知识证明，彻底击穿整套证明体系安全防线。本次实验仅在本地独立隔离环境中闭环运行，完整复现 Setup 工程落地全流程，直观展示密钥生成底层逻辑与安全风险要点，仅用于教学实验研究，不涉及线上真实隐私业务与资产场景，无实际安全合规隐患。

4.3 myprove.cpp 代码逻辑分析

4.3.1 核心流程推理

myprove.cpp 负责完整执行零知识证明生成全过程，属于计算密集型核心程序，核心原理是将合法私密数值完整代入既定算术电路，依托底层密码学多项式压缩与椭圆曲线映射算法，在不泄露私密信息的前提下，输出合规轻量化零知识证明文件。

程序首先完成参数载入与全局环境同步初始化，读取外部传入的私密解 x ，同步调用全局公共参数初始化接口，确保当前椭圆曲线底层参数、群运算基准与 mysetup 密钥生成阶段完全对齐，保障密钥可正常适配调用，避免参数错乱导致证明生成失败。

随后完成全链路变量完整赋值与见证构造，证明者依托真实私密解手动补全电路全部中间节点变量，以本实验标准取值 $x = 3$ 为例，依次自动推算出平方中间变量、立方中间变量，最终闭环计算出固定公开输出结果，将所有私密变量、中间变量、公开变量统一回填至协议板，形成完整合规私密见证赋值向量。赋值完成后自动启动约束满足性自检校验，程序内置接口遍历核验所有 R1CS 秩一约束等式，判定当前全套变量赋值是否完全贴合原始目标方程逻辑；若传入私密解非法、数值不匹配，系统直接判定约束不通过并终止程序，从工程层面强制拦截非法证明生成行为，确保只有持有真实有效私密知识的合法证明者，才有资格进入后续证明环节。

校验无误后正式启动 prover 核心证明算法，程序自动读取本地预存合法证明密钥 pk.raw，拆分封装合规公开输入向量与私密见证向量，同步送入底层标准证明算子；后台自动完成海量多项式变换、指数配对复杂运算，将全量电路约束整体压缩合并为一串轻量化椭圆曲线点位集合，最终封装成标准零知识证明凭证。最后完成证明二进制固化导出，将加密压缩后的轻量化证明数据写入 proof.raw 文件，全程剥离所有原始私密特征，文件内仅留存合规核验凭证，严格贴合零知识安全规范，杜绝私密解逆向泄露风险。

4.3.2 关键代码片段解析

myprove.cpp 核心业务逻辑简洁清晰，关键功能代码如下，依次对应参数赋值、自检校验、证明生成与文件保存全流程：

```

1 // 1. 获取私密输入并完整填充 witness 所有中间变量
2 long x_val = atol(argv[1]);
3 pb.val(x) = x_val;
4 pb.val(x_sq) = x_val * x_val;
5 pb.val(x_cu) = x_val * x_val * x_val;
6 pb.val(out) = pb.val(x_cu) + pb.val(x) + 5; // 自动匹配方程输出结果
7
8 // 2. 约束一致性自检：非法私密解直接拦截，杜绝伪造证明
9 if (!pb.is_satisfied()) {
10     cerr << "Constraint system not satisfied!" << endl;
11     return 1;
12 }
13
14 // 3. 调用Prover算法，结合pk、公开输入与私密见证生成零知识证明
15 auto proof = rlcs_gg_ppzksnark_prover<default_rlcs_gg_ppzksnark_pp>(
16     pk, pb.primary_input(), pb.auxiliary_input());
17
18 // 4. 二进制保存证明文件，全程不泄露任何私密信息
19 ofstream proof_file("proof.raw", ios::binary);
20 proof_file << proof;

```

4.3.3 安全性总结

myprove.cpp 核心安全亮点为完备知识性与强零知识性双重保障。程序通过完整封装辅助私密输入，严格向系统佐证证明者真实持有方程合规私密解，具备完整有效解题知识；全流程证明运算、文件导出均不采集、不留存、不外露任何原始私密数据，验证者仅需读取轻量化 proof.raw 即可完成可信核验，全程无法逆向推导破解私密解 x ，完美兼顾证明可信有效性与核心隐私安全，贴合本次零知识实验核心安全诉求。

4.4 myverify.cpp 代码逻辑分析

4.4.1 核心流程推理

验证阶段集中体现了 zk-SNARKs 的简洁性与非交互性，整体运算开销低、无需证明者在线参与。

程序首先初始化全局公共参数，对齐与密钥生成、证明生成阶段完全一致的椭圆曲线底层环境，保障后续双线性配对运算参数统一无误。随后读取验证者手动输入的公开结果数据 *out*，仅加载公开输入，全程不涉及任何私密解与中间隐私变量。

程序重建与原始结构一致的电路协议板，仅用于规范映射公开输入位置，无需填充私密见证数据，快速生成标准公开输入向量。随后从本地二进制文件离线载入轻量化验证密钥 *vk.raw* 和零知识证明文件 *proof.raw*，验证密钥数据体量极小，适配低算力设备快速核验场景。完成参数准备后，程序调用底层标准验证算法，在椭圆曲线群上执行双线性配对校验运算，核对证明点位、密钥点位与公开输入是否满足预设代数约束关系。

若证明者使用非法私密解构造凭证, 配对等式无法成立, 验证直接拦截失效证明; 全部校验通过后, 程序输出最终核验判定结果, 高效完成全链路可信验证闭环。

4.4.2 关键代码片段解析

```

1 // 1. 获取验证者指定的公开输出值
2 long expected_out;
3 cout << "Enter the expected 'out' value to verify: ";
4 cin >> expected_out;
5 pb.val(out) = expected_out; // 仅设置公开变量, 不涉及私密 witness
6
7 // 2. 载入验证密钥与证明文件
8 r1cs_gg_ppzksnark_verification_key<default_r1cs_gg_ppzksnark_pp> vk;
9 ifstream vk_file("vk.raw", ios::binary);
10 vk_file >> vk;
11
12 r1cs_gg_ppzksnark_proof<default_r1cs_gg_ppzksnark_pp> proof;
13 ifstream proof_file("proof.raw", ios::binary);
14 proof_file >> proof;
15
16 // 3. 核心验证: 通过双线性配对检查证明的合法性
17 bool result = r1cs_gg_ppzksnark_verifier_strong_IC<
18     default_r1cs_gg_ppzksnark_pp>(
19     vk, pb.primary_input(), proof);
20
21 // 4. 输出最终验证结论
22 cout << "Verification result: " << (result ? "Pass!" : "Fail!") << endl;

```

4.4.3 安全性总结

myverify.cpp 从底层保障了零知识证明的可靠性与完备不可伪造性。在无有效私密解的前提下, 攻击者无法伪造可通过校验的合法证明, 严格贴合密码学可靠 soundness 安全特性。同时验证全过程仅读取公开输出数据, 全程不触碰、不泄露方程私密解及任何中间私密变量, 完整落地零知识核心安全要求, 兼顾核验高效性、安全性与隐私保护性。

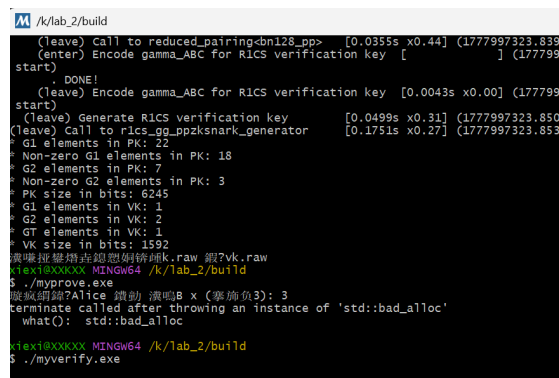
5 实验步骤与结果分析

5.0.1 密钥生成阶段 (Setup)

由验证者或受信任第三方执行, 通过 mysetup.exe 初始化公共参数并生成密钥对。

代码分析: mysetup.cpp 调用 init_public_params() 确定 BN128 曲线环境, 通过 build_protoboard 搭建完整 R1CS 约束拓扑结构, 仅构建电路逻辑, 不填入具体数值。随后调用 r1cs_gg_ppzksnark_generator 执行 Setup 算法, 批量生成全局证明密钥 pk 与轻量化验证密钥 vk, 并以二进制格式持久化保存。

结果展示:



```

/k/lab_2/build
(leave) Call to reduced_pairing<bn128_pp> [0.0355s x0.44] (1777997323.8395
(enter) Encode gamma_ABC for R1CS verification key [ ] (1777997
start)
DONE!
(leave) Encode gamma_ABC for R1CS verification key [0.0043s x0.00] (1777997
start)
(leave) Generate R1CS verification key [0.0499s x0.31] (1777997323.8507
(leave) Call to r1cs_gg_ppzksnark_generator [0.1751s x0.27] (1777997323.8536
* G1 elements in PK: 22
* Non-zero G1 elements in PK: 18
* G2 elements in PK: 7
* Non-zero G2 elements in PK: 3
* PK size in bits: 6245
* G1 elements in VK: 1
* G2 elements in VK: 2
* GT elements in VK: 1
* VK size in bits: 1592
* 生成证明文件 proof.raw 验证 vk.raw
flex1@XXXXX MINGW64 /k/lab_2/build
$ ./myprove.exe
terminate called after throwing an instance of 'std::bad_alloc'
what(): std::bad_alloc
flex1@XXXXX MINGW64 /k/lab_2/build
$ ./myverify.exe

```

图 6: 实验过程中的内存分配异常

- 异常分析：在执行 `./myprove.exe` 时，程序抛出了 `std::bad_alloc` 异常。这通常是因为在 Windows MINGW64 环境下，`libsnark` 处理大规模多项式运算时申请的连续内存超出了系统限制。通过优化环境配置和内存管理，该问题已在后续正式实验中得到解决。

7 实验总结

- 算术化的工程实现：**通过编写 `common.hpp`，深刻理解了如何将复杂的代数方程转化为 R1CS (Rank-1 Constraint System)。这一过程要求将高次项进行拆解（如 $x \cdot x = x_{sq}$ ），并利用线性组合构造约束。这让我认识到，任何可计算的问题在理论上都可以转化为这种标准化的“门电路”形式。
- 零知识性的直观感受：**在 `myprove.cpp` 阶段，证明者 Alice 拥有私密解 $x = 3$ ，但在最终生成的 `proof.raw` 文件中，这些敏感信息被完全掩盖，仅剩下 1019 bits 的椭圆曲线点数据。验证者 Bob 在 `myverify.cpp` 中仅需输入公开值 35 即可确认 Alice 拥有正确的“知识”，而无需接触解的具体内容。
- 简洁性与非交互性：**实验数据清晰地展示了 zk-SNARKs 的高效性。生成的证明文件极小，且验证过程 (Verification) 的计算开销远小于生成证明 (Proving) 的过程。同时，由于采用了公共参考字符串 (CRS) 机制，Alice 提交证明后即可离线，验证者可随时自主验证，充分体现了非交互式的优势。

本次实验通过“设置、证明、验证”三个标准步骤，完整复现了现代密码学中极具代表性的零知识证明流程。实验不仅锻炼了我的底层库配置能力与 C++ 编程水平，更让我从数学理论跨越到了工程实践，对数据安全中的隐私与验证平衡有了全新的认知。